Backend route excerpt (continued) *app.py*

```
    # run; without this, leftover trajectory data from a previous run can mix
    # with the current run and make predicted aircraft "trace back" their route.
    flight_store.reset_for_new_session()

if not matrix_csv_enabled:
    try:
        Path(matrix_csv_path).unlink(missing_ok=True)
    except Exception as exc:
        app.logger.warning("Could not remove matrix CSV file: %s", exc)

if os.getenv("FLIGHT_ICAO_EXCEL_ENABLED", "true").lower() == "true" and not flight_store.icao_excel_enabled:
    msg = "ICAO Excel logging is enabled but unavailable. Install openpyxl in the same Python environment."
    app.logger.warning(msg)
    print(f"WARNING: {msg}")


def _build_storage_rows(raw_rows, normalized_rows):
    """
    Preserve normalized core fields while keeping optional telemetry from raw rows
    (e.g. verVelocityCMS/sourceIp/squawk) for logging.
    """
    merged_rows = []
    total = max(len(raw_rows), len(normalized_rows))
    for idx in range(total):
        raw = raw_rows[idx] if idx < len(raw_rows) and isinstance(raw_rows[idx], dict) else {}
        norm = normalized_rows[idx] if idx < len(normalized_rows) and isinstance(normalized_rows[idx], dict) else {}
        rec = dict(raw)
        rec.update(norm)
        merged_rows.append(rec)
    return merged_rows

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/aircraft")
def aircraft():
    # Get data from source
    data = get_aircraft_data()

    # If data is a dict with an 'aircraft' key, get that list.
    # Otherwise, assume data itself is the list.
    raw_list = data.get("aircraft", []) if isinstance(data, dict) else data
    storage_input_list = (
        data.get("aircraft_events", data.get("aircraft_raw", raw_list))
        if isinstance(data, dict)
        else raw_list
    )

    # Process the list
    processed_list = [preprocessor.preprocess(ac) for ac in raw_list]

    # Pass through a few extra telemetry fields the UI can use for a richer
    # flight-detail panel (vertical speed, squawk, source feed) without
    # changing the normalized schema used elsewhere.
    for idx, source_row in enumerate(raw_list):
        if idx >= len(processed_list) or not isinstance(source_row, dict):
            continue
        for key in ("verVelocityCMS", "squawk", "sourceIp"):
            value = source_row.get(key)
            if value not in (None, ""):
                processed_list[idx][key] = value

    processed_storage_list = _build_storage_rows(
        storage_input_list,
        [preprocessor.preprocess(ac) for ac in storage_input_list],
    )
```